

Anatomy of a Speedup: EAGLE-3 Speculative Decoding on Qwen3

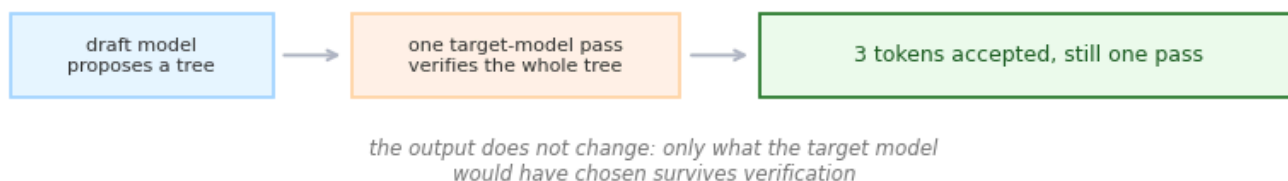
By [Georgy Mamarin](#) · a measured walkthrough of [EAGLE-3](#) (Yuhui Li, Fangyun Wei, Chao Zhang, Hongyang Zhang — NeurIPS 2025)

A language model writes one token at a time, and every token costs a full pass through all of its weights. Speculative decoding works around that: a small draft model proposes several next tokens in advance, the big target model checks them all in a single pass and keeps the ones that match its own choice. The output text is the same — only getting it becomes faster.

ordinary generation



speculative decoding



Top row: ordinary generation, five passes for five tokens. Bottom row: one target-model pass returns three. How much of a win that is comes down to one thing: how often the draft model is right.

The paper reports up to 6.5× on its own setups. I re-ran the method on Qwen3-1.7B with a free Kaggle T4: about 2.5× on math and code, about 2.3× across the paper's four English benchmarks — and about 0.95× on Russian prompts, where the method made generation slower than the plain loop. Both outcomes follow from a single formula for the speedup. I derive it in [§1](#), and every section after that pins down one of its terms.

Run All needs a T4 GPU. Weights load from the attached Kaggle models — internet is used only for one pip install and cloning the EAGLE repo. The full run takes about 35 minutes, or about 10 with `QUICK = True`.

What's in here

- EAGLE-3 running on Qwen3-1.7B with the official SafeAIlab code and attached Kaggle weights — [§4](#).

- Vanilla autoregressive vs EAGLE-3, measured on five prompt sets with equal work per engine — §5.
- The draft tree built, dissected and timed: shape ablation, cycle phases, losslessness checks — §6–§10.
- The same protocol re-run on a target twice the size, Qwen3-4B — §11.

What you'll learn

1. Why verifying a whole draft tree costs about as much as generating a single token — measured, not quoted from a spec sheet.
2. What an ordinary generation step really costs: four times the theoretical minimum, and why part of the method's win is amortizing that overhead.
3. What the draft tree looks like from the inside, and how much branching adds over a plain chain.
4. Which tree shape is optimal — one knob at a time, with repeats, instead of turning three knobs at once.
5. How to check the method doesn't change the output: token-by-token under greedy decoding, statistically under sampling.
6. When the method hurts: off the draft head's training domain it loses to the plain loop.
7. Whether a bigger target really amortizes the draft head better — the common intuition, put to a measurement on Qwen3-4B.

Contents

- [0. Run parameters](#)
- [1. The arithmetic of decoding](#)
- [2. The KV cache: what memory costs, and why a tree can live there](#)
- [3. What EAGLE-3 actually changes — and what shows up in the weights](#)
- [4. Three ways to generate with one model](#)
- [5. Benchmarks from the paper](#)
- [6. The draft tree: anatomy of a single step](#)
- [7. Tree shape: two one-knob sweeps](#)
- [8. Where the cycle spends its time](#)
- [9. Temperature: where the method's main guarantee is tested](#)
- [10. Matching plain generation at \$T = 0\$](#)
- [11. The same method, a bigger target](#)
- [12. What didn't work](#)
- [13. Takeaways](#)
- [14. Run it yourself, further reading, licenses](#)

0. Run parameters

The four lines below are everything you touch when forking this notebook to try another combination. The rest assembles itself around them: the weights load, and the prompt sets, tables and figures rebuild with the same settings.

- **BASE_MODEL** — the target model whose generation is being sped up. Resolved from the attached Kaggle model, with a Hub fallback off-Kaggle.
- **EA_MODEL** — the draft head trained for this exact target. Ready-made heads for other targets are listed in the EAGLE README, linked from §14.
- **QUICK** — a ~10-minute run instead of ~35: fewer questions per set, fewer repeats per measurement, and the scaling probe (§11) is skipped entirely — it needs a second pair of models loaded.
- **LANG_SET** — the prompt set treated as out-of-domain for the draft head. It must match a key of the **BENCH** dict in §5 (add your own set there); §5 uses it to show what happens when the head was trained on different data.

```
import glob, os

def attached(pattern, hub_id):
    """Weights come from attached Kaggle models; fall back to the Hub elsewhere."""
    hits = sorted(glob.glob(os.path.join(pattern, "config.json")))
    return os.path.dirname(hits[-1]) if hits else hub_id

BASE_MODEL = attached("/kaggle/input/models/qwen-lm/qwen-3/transformers/1.7b/*",
                      "Qwen/Qwen3-1.7B")
EA_MODEL    = attached("/kaggle/input/models/georgymamarin/qwen3-1-7b-eagle3-draft-head",
                      "AngelSlim/Qwen3-1.7B_eagle3")
QUICK       = False
LANG_SET    = "Russian"
print(BASE_MODEL); print(EA_MODEL)    # confirm the weights resolved to /kaggle/input

/kaggle/input/models/qwen-lm/qwen-3/transformers/1.7b/1
/kaggle/input/models/georgymamarin/qwen3-1-7b-eagle3-draft-head/pytorch/fp16/1
```

The cell below prepares the environment: installs the pinned **transformers**, clones the EAGLE repository at a fixed commit and checks the runner actually got the GPU everything here was measured on. It also prints versions: Kaggle's base image updates silently and breaks old notebooks.

```
GPU: Tesla T4 (sm_75)
torch 2.10.0+cu128 | transformers 4.53.1
EAGLE imported, commit cb7e0841fe0c
python 3.12.13 | numpy 2.0.2 | pandas 2.3.3
```

1. The arithmetic of decoding

Autoregressive generation yields one token per pass, and each pass makes the GPU read every weight of the model for the sake of that one token. That comes to roughly one operation per byte read, while the card is built for about 200:

$$\frac{2 \text{ op/param}}{2 \text{ byte/param}} = 1, \quad \frac{65 \text{ TFLOPS}}{320 \text{ GB/s}} \approx 200$$

On the left, the arithmetic intensity of fp16 decoding; on the right, the T4's datasheet balance. Decoding sits two hundred times below the break-even point: a step takes exactly as long as pumping the weights through the memory bus, and the compute units idle. Generation is memory-bound, not compute-bound.

The key consequence: if step time is set by reading weights, then processing several tokens instead of one barely changes the cost, because the weights are read once either way. Verifying k drafts should be about as cheap as generating a single token.

That claim is usually quoted. Here it gets measured — along with the boundary where "almost free" ends.

Loading the target model and the draft head. Both are small: the target is 3.2 GB in half precision, the head another 0.27 GB, so everything fits comfortably into the T4's 16 GB.

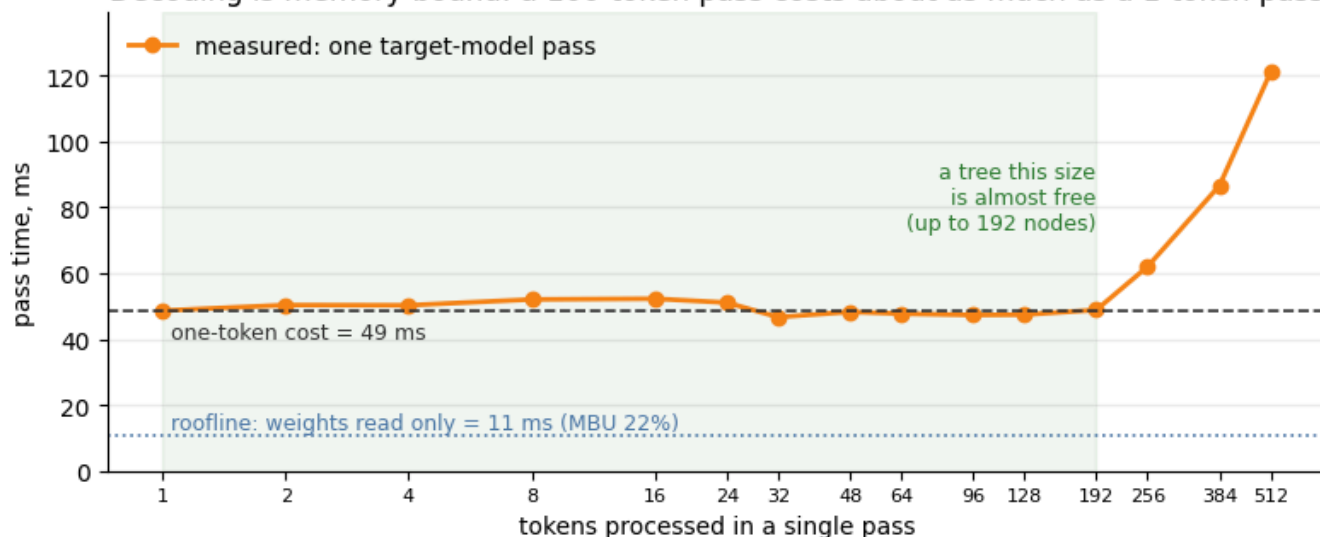
```
loaded in 36 s
target model: 1.72B params, 28 layers, 3.20 GB in fp16
draft head: 137M (8.0% of the target)
```

First measurement: how long one target-model pass takes when it is fed several tokens at once instead of one. If the reasoning above holds, the curve should be flat — the weights are read once no matter how many tokens are processed.

tokens	ms	ms/token	vs n=1
1	48.6	48.617	1.00x
2	50.3	25.158	1.03x
4	50.2	12.562	1.03x
8	52.0	6.503	1.07x
16	52.2	3.265	1.07x
24	51.1	2.127	1.05x
32	46.7	1.458	0.96x
48	48.2	1.005	0.99x
64	47.7	0.745	0.98x
96	47.3	0.493	0.97x
128	47.4	0.370	0.97x
192	48.8	0.254	1.00x
256	61.9	0.242	1.27x
384	86.5	0.225	1.78x
512	121.0	0.236	2.49x

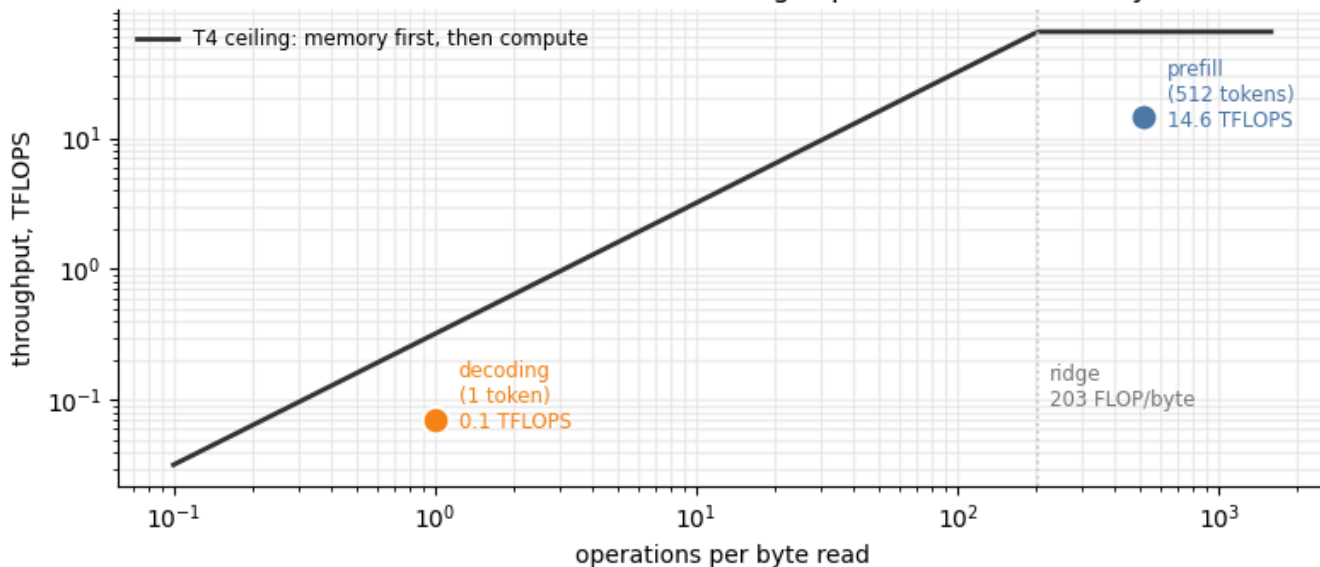
```
one token: 48.6 ms
roofline prediction (weights read only): 10.8 ms
=> memory-bandwidth utilization (MBU): 22%
=> a pass gets less than 25% dearer all the way to 192 tokens
```

Decoding is memory-bound: a 100-token pass costs about as much as a 1-token pass



Takeaway. The curve stays flat to just under 192 tokens: a 100-token pass costs about as much as a 1-token pass. That headroom is what speculative decoding runs on. The second thing to notice is the gap to the dashed line: an ordinary step costs four times the theoretical minimum, so the plain loop has slack of its own — and part of the upcoming speedup will come from there.

Where we live on the roofline: decoding is pinned to the memory limit



decoding (1 token)	ops per byte	1.0		0.1 TFLOPS of 65
prefill (512 tokens)	ops per byte	512.0		14.6 TFLOPS of 65

Takeaway. Decoding a single token lives on the left of the diagram, where throughput is capped by the memory bus and the compute units idle. Prefill at 512 tokens sits two orders of magnitude to the right, up against a different ceiling. Speculative decoding pushes the generation step rightward: more useful work for the same bytes read.

The formula this whole walkthrough reduces to

Acceptance length τ is the average number of tokens a cycle yields per target-model pass: the accepted drafts plus the bonus token, so in the code it is `accept_length` + 1. Plain generation has $\tau = 1$ by definition: one pass, one token.

The gain then reads:

$$S \approx \frac{\tau \cdot t_{\text{step}}}{t_{\text{verify}} + (d+1) \cdot t_{\text{draft}}}$$

Notation:

- S — speedup over plain generation;
- τ — acceptance length;
- d — tree depth;
- t_{step} — time of an ordinary generation step;
- t_{verify} — time to verify the whole tree in one target-model pass;
- t_{draft} — time of one draft-model step.

The numerator is what a cycle brings in; the denominator is what it costs. The rest of the notebook measures each term separately: t_{verify} here and in §8, t_{draft} in §7 and §8, τ in §5. Every result below plugs back into this formula, including the one where the speedup lands under 1.

The baseline. The measured step costs about four times the roofline prediction: the memory bus runs at roughly a quarter of its bandwidth. The difference is implementation overhead — attention without flash kernels, op-by-op CUDA launches from Python, no CUDA graphs. Speculative decoding divides that overhead by τ accepted tokens, so *part* of the speedup measured below amortizes the baseline's inefficiency rather than beating the memory wall. On a tuned stack (vLLM, SGLang, CUDA graphs) an ordinary step sits closer to the memory limit and speculation gains less: independent vLLM measurements report 1.3–2× where the authors' own frameworks show 4–6× ([*"Performance or Illusion?"*, 2601.11580](<https://arxiv.org/abs/2601.11580>)).

One more boundary, without which the numbers read wrong. Everything measured here is latency at batch = 1. Production cares about throughput, and that picture is different: at a

large batch the compute units are busy anyway, nothing is left to verify drafts with, and the gain collapses. The paper itself shows this: 4–6× at batch = 1 against 1.38× in SGLang at batch 64. Speculative decoding treats decode and decode only — prefill lives further right on the roofline and needs no treatment.

2. The KV cache: what memory costs, and why a tree can live there

The measurement above showed that a tree gets verified almost for free. What remains is where that tree lives between steps, and why it can live there at all.

Without an attention cache, generation would be quadratic: every new token would recompute keys and values for the whole prefix. The KV cache stores them and makes a step linear in context length. The price is memory, and it can be counted exactly.

For this model: 28 layers, 8 KV heads against 16 attention heads (GQA, twice as frugal as full attention), head size 128, 2 tensors (K and V), 2 bytes per number.

Speculative decoding adds a requirement plain generation does not have. One pass verifies a whole tree, but only one path in it gets accepted. So the cache must do three things: let each node attend only to its ancestors (**tree attention** — a mask, not the usual causal one), hold K and V for every node of the tree at once, and, after verification, keep just the winning path, dropping the other branches without any recompute.

EAGLE does this with no allocations in the loop: a buffer for the whole context is allocated once (`initialize_past_key_values`), and when a candidate is accepted its positions are copied into place inside that same buffer (`dst.copy_(tgt)` in `update_inference_inputs`), after which the length pointer advances. Correctness rests on one fact: each node's K and V were computed against its ancestors only, so for the accepted path they are exactly what plain sequential decoding would have produced. Rejected branches leave no trace.

Counting what the cache costs on this model, and how much of it a draft tree occupies.

layers 28 | KV heads 8 (attention heads 16, i.e. GQA x2) | head_dim 128

KV per token: 112 KB

context 512 → 56 MB (1.7% of model weights)

context 2048 → 224 MB (6.8% of model weights)

context 8192 → 896 MB (27.3% of model weights)

context 40960 → 4480 MB (136.5% of model weights)

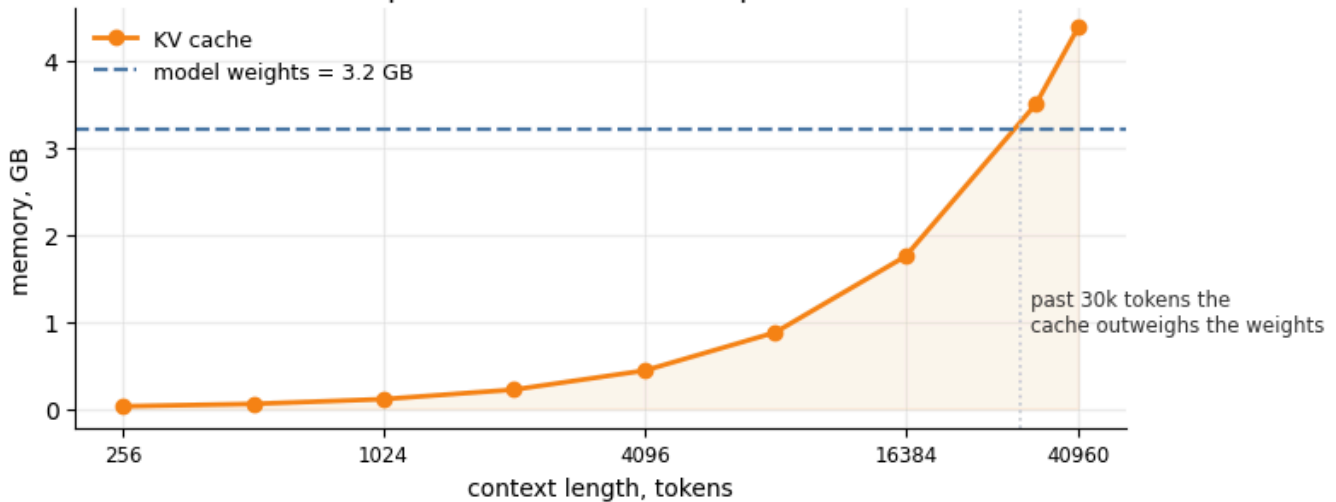
a 32-node tree holds 3.5 MB of cache — under 0.11% of the weights

a 96-node tree holds 10.5 MB of cache — under 0.32% of the weights

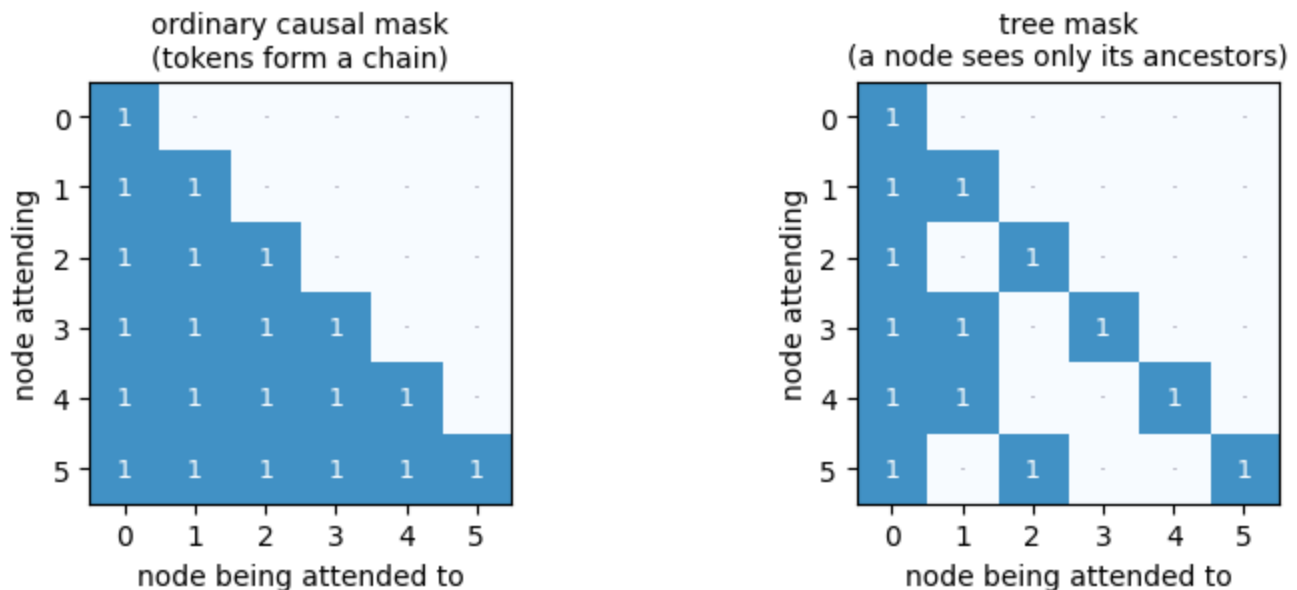
a 256-node tree holds 28.0 MB of cache — under 0.85% of the weights

model weights in fp16: 3.20 GB | GPU allocated now: 4.07 GB

The price of context: 112 KB per token



Takeaway. At short context the KV cache barely registers, but it grows linearly and at Qwen3's 40k-token limit it overtakes the model weights themselves. Against that, the draft tree is free: a hundred nodes take a few megabytes. This is why speculative decoding costs nothing in memory — it spends what is abundant to save what is scarce. The flip side: at long context the arithmetic of §1 shifts, because a step now drags the cache along with the weights, so both plain generation and tree verification get dearer together. Every measurement in this walkthrough is taken on short prompts, where the cache does not yet matter.



Tree: 1<-0 2<-0 3<-1 4<-1 5<-2

Nodes 3 and 5 sit on different branches, and in the tree mask they cannot see each other — though a causal mask would let node 5 see node 3 merely for being earlier.

Takeaway. A causal mask lets every token attend to everything to its left, which is right for a chain. A tree cannot afford that: nodes from different branches share one tensor but represent mutually exclusive continuations. The tree mask restricts each node to its ancestors; without it the branches would bleed into each other and verification would mean nothing.

3. What EAGLE-3 actually changes — and what shows up in the weights

The cache can hold a whole tree and keep just its accepted path. Now: how the third version of the method differs from the first two, and which of those differences show up not in the paper but in the code.

EAGLE-3 is the third paper of a series: EAGLE-1 (ICML'24), EAGLE-2 (EMNLP'24), EAGLE-3 (NeurIPS'25). Its headline 6.5× is the best single cell of the results table (HumanEval on Vicuna-13B); the per-model averages are more modest, 5.51 on Vicuna-13B and 4.44 on LLaMA-3.1-8B, all under greedy decoding at batch = 1 inside the authors' own framework. The same paper carries a second number that measures something else entirely: in SGLang at batch 64, throughput improves by 1.38×.

EAGLE-1 ([2401.15077](#)) noticed that predicting the next token with a small model is hard, while predicting a **feature** — the target model's top-layer hidden state — is noticeably easier: features are more regular than tokens. Its draft model is a single transformer layer that takes the feature `f_t` and the embedding of the already-known token `x_{t+1}`, then continues the feature chain autoregressively. The second input removes sampling uncertainty: the draft knows which token actually came out.

EAGLE-2 ([2406.16858](#)) made the draft tree dynamic. The tree shape used to be fixed; now nodes expand by the draft model's confidence, and the `total_token` best candidates by cumulative log-probability make it into the tree. Where the draft is confident the tree grows deep; where it hesitates, it grows wide.

EAGLE-3 ([2503.01840](#)) makes three changes:

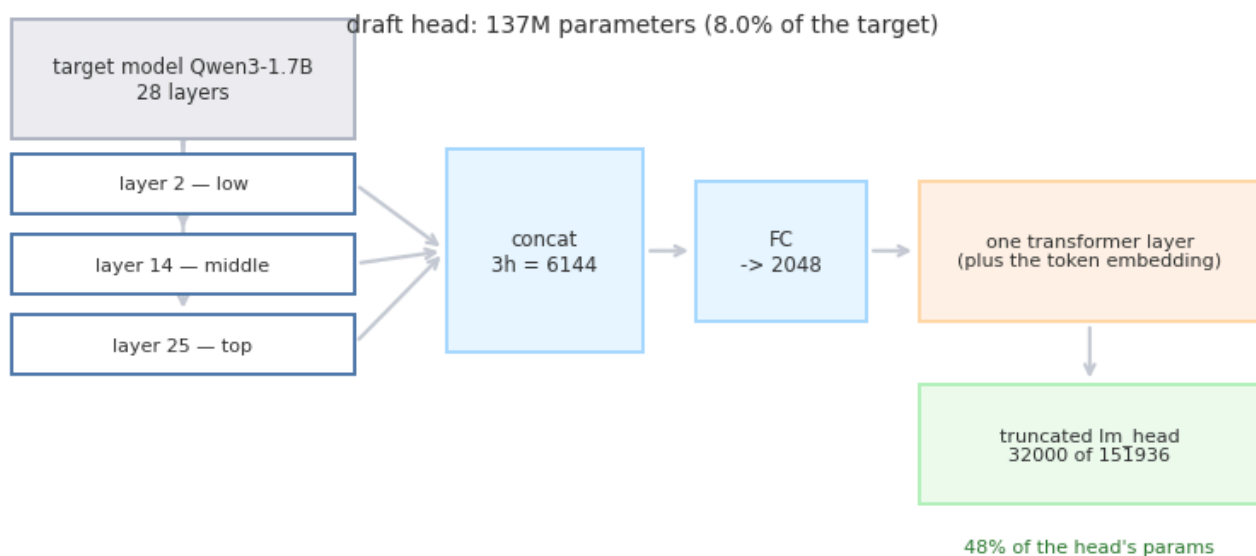
1. **The draft model stops predicting the feature.** The loss used to carry an `l_fea` term — a regression onto the target's feature. It acted as a regularizer and hit a ceiling: draft quality stopped improving with more training data. EAGLE-3 trains the draft to predict tokens only, and gets the paper's main result: acceptance length starts scaling with data. A subtlety: feature-space autoregression is still there — the draft still feeds its hidden state into its next step. What is gone is the loss that forced that state to match the target's feature.
2. **Three-level fusion.** Instead of one top-layer feature, a concatenation of hidden states from a low, a middle and a top layer, compressed by a $3h \rightarrow h$ linear. The top feature is tuned to the

very next token and loses information useful several steps ahead.

3. **Training-time test.** At generation time the draft walks a chain of its own outputs, which it never saw in training, and drift accumulates. EAGLE-3 unrolls the draft on its own predictions during training, imitating generation. The closest ancestor of the idea is HASS ([2408.15766](#)).

Three things on that list are not spelled out in the paper but are immediately visible in the loaded weights: which exact layers the features come from, how far the draft vocabulary is truncated, and which parameter is left dead in the code.

- 1) Target layers feeding the fusion (in the code: `idx == 2, L//2, L-3`):
low 2 | middle 14 | top 25 of 28 layers
fc: (2048, 6144) – compresses 3h → h
- 2) The truncated draft vocabulary is not a detail but a condition of viability:
target vocab 151936, draft vocab 32000
draft lm_head: 66M params = 48% of the whole head
with the full vocab the head would weigh 382M = 22% of the target (instead of 8.0%)
on a target this small, EAGLE-3 would not pay off without the truncation
- 3) How the vocabularies connect (d2t stores OFFSETS, not indices):
d2t: (32000,) torch.int64, applied as token = i + d2t[i]
t2d: (151936,) torch.bool – coverage mask over the target vocab
target tokens covered: 32000 of 151936
the draft cannot propose a token outside its vocab, but the target is free to emit one as the bonus token – correctness is unaffected
- 4) The threshold parameter: lives in the constructor, unused in the EAGLE-3 branch.
self.threshold is set in `__init__`, mentions inside `topK_genrate: 0`
(in `cnets1.py` – the EAGLE-1/2 draft – it still worked as a pruning threshold)



three-level fusion is the EAGLE-3 change you can see in the weights

Takeaway. The draft is far simpler than its target: three hidden-state taps, a linear squeeze, one transformer layer and an output vocabulary layer. Nearly half of all its parameters sit in that layer, truncated to 32k tokens instead of the full vocabulary: without the truncation the draft model would weigh a fifth of its target and would never pay for itself at this scale.

4. Three ways to generate with one model

First, code and weights. The code is the official [SafeAILab/EAGLE](#) repository at commit `cb7e0841` — `eagenerate`, `naivegenerate`, the tree construction and the benchmark files all come from there. The draft head is the one that same repository lists in its weights table for Qwen3-1.7B: [AngelSlim/Qwen3-1.7B_eagle3](#), attached here as a Kaggle model. The table marks it as non-authorial — it was trained by the [AngelSlim](#) team at Tencent: the EAGLE authors' own ready-made heads cover only Vicuna and LLaMA. This matters more than it sounds when reading the numbers: the head was trained on English dialogue, and off that domain it behaves very differently (§5).

All three ways run the same weights: one model, three loops calling it.

generate from transformers — what an ordinary user gets. An honest external reference, but it uses a different attention backend and a different cache, so it does not isolate the effect of speculation.

naivegenerate — the plain autoregressive loop from the EAGLE repository: the same modified model and the same static KV buffer as the speculative loop. This is the baseline the paper computes its speedups against, and the one that isolates the method's contribution: between it and `eagenerate` there is exactly one difference — speculation.

eagenerate — EAGLE-3 itself.

Every number below is reported against both baselines. The uncomfortable part: on this card `naivegenerate` runs about 10% slower than stock `generate` — it has no flash attention — so a speedup measured against it flatters the method slightly. Hence two columns in the final table.

One more methodological choice. All three stop at EOS, and their texts diverge (§10 covers why), so tokens-per-second would be computed over different work. To keep the comparison honest, the measurements suppress the end-of-sequence token and force every loop to emit the same number of tokens.

Below is the shared measurement harness: one way to build a prompt, a stopwatch around generation, and a tree-shape switch. The EOS suppression from the previous paragraph lives here too.

5. Benchmarks from the paper

The three loops are ready; what they need is an honest task set. Hand-written prompts are the weak spot of any speed claim — they can always be tuned toward a convenient answer. Fortunately, the EAGLE repository ships the very sets the paper reports on: MT-Bench (dialogue), GSM8K (arithmetic word problems), HumanEval (code), Alpaca (instructions) — 80 questions each. I take the first five of each: a full 320-question sweep does not fit into a free session.

The fifth set is five Russian prompts of my own — not a public benchmark. The draft head was community-trained on English data, and I want to see the method operate outside its domain. As it turns out, that regime produces the most instructive result of the whole walkthrough.

Call order within a question is fixed — stock `generate`, then `naivegenerate`, then `eagenerate`; no randomization. All three run back-to-back on the same question, so GPU thermal drift hits them almost equally: the run-to-run spread of plain generation is a fraction of a percent (§7 prints it explicitly).

The numbers in this section are taken at the starting tree shape `60/7/10` (total_token / depth / top_k) set at model load. §7 will show this shape is not optimal: a shallower tree comes out roughly 10% faster (§7 prints the sweep and the winning shape; the exact figure moves a few points between runs). I did not rerun the full benchmark at the chosen shape: a Kaggle session is finite, and the section's conclusion — a two-and-a-half-fold gap between domains — is a ratio of two speedups measured at the same shape, so a shift common to both cancels out of it.

Assembling the five sets: four from the paper's repository plus my Russian one.

```
MT-Bench  : 5 questions | first: 'Compose an engaging travel blog post about a recent
trip to Hawa'
GSM8K      : 5 questions | first: "Darrell and Allen's ages are in the ratio of 7:11.
If their tota"
HumanEval  : 5 questions | first: 'Complete the code I provided.\n\ndef below_threshol
d(l: list, t: i'
Alpaca     : 5 questions | first: 'What would be the best type of exercise for a perso
n who has art'
Russian    : 5 questions | first: 'Объясни в трёх предложениях, почему небо голубое.'
```

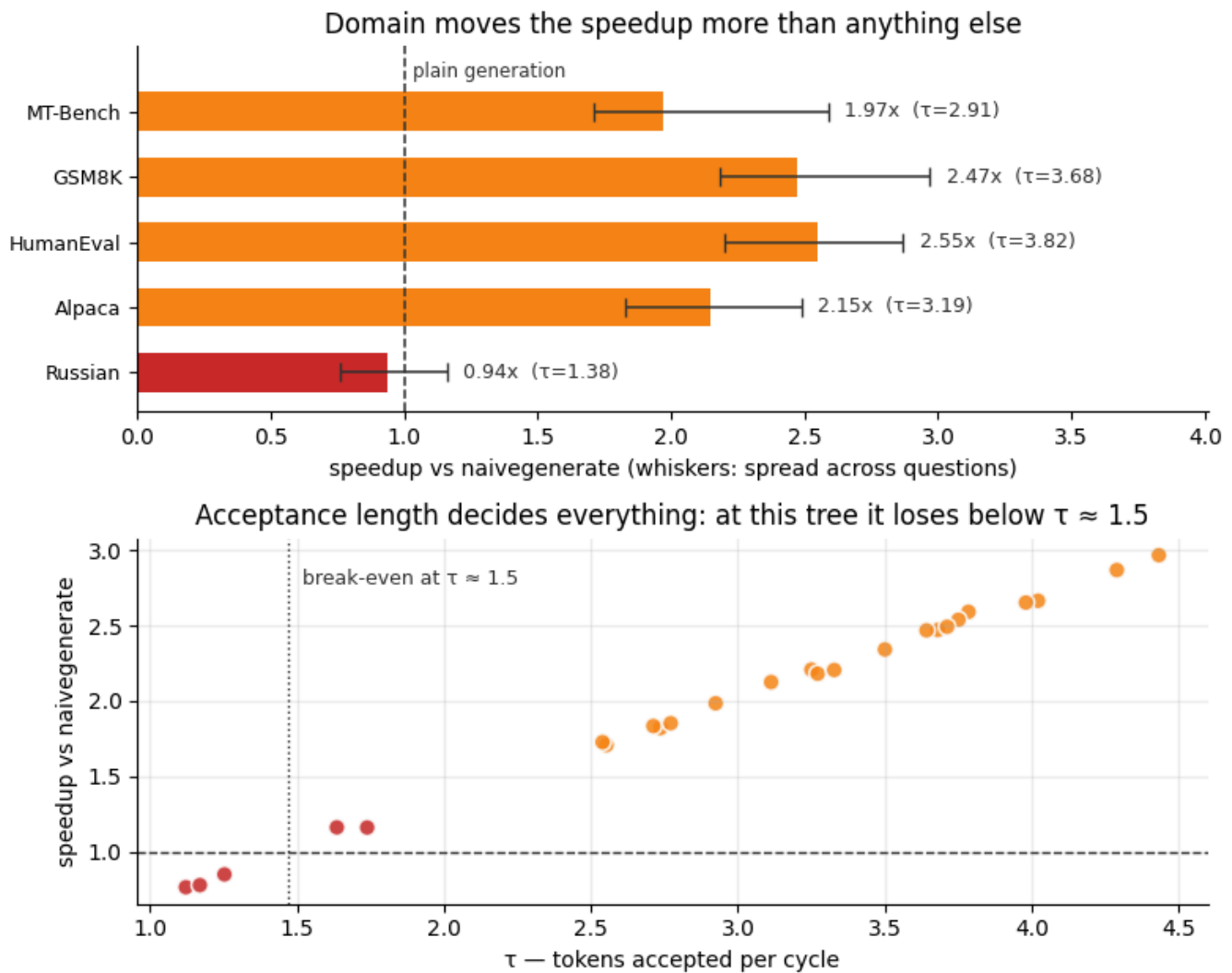
This is where §5's numbers come from: every loop emits a fixed number of tokens, and time is divided by that.

MT-Bench	q0: naive	46.9 ms/tok		eagle	27.4 ms/tok		τ 2.55
MT-Bench	q1: naive	47.0 ms/tok		eagle	18.2 ms/tok		τ 3.78
MT-Bench	q2: naive	46.3 ms/tok		eagle	23.3 ms/tok		τ 2.92
MT-Bench	q3: naive	49.3 ms/tok		eagle	27.1 ms/tok		τ 2.74
MT-Bench	q4: naive	48.0 ms/tok		eagle	27.8 ms/tok		τ 2.54
GSM8K	q0: naive	47.5 ms/tok		eagle	18.7 ms/tok		τ 3.75
GSM8K	q1: naive	47.6 ms/tok		eagle	21.6 ms/tok		τ 3.25
GSM8K	q2: naive	46.8 ms/tok		eagle	15.8 ms/tok		τ 4.43
GSM8K	q3: naive	46.6 ms/tok		eagle	21.4 ms/tok		τ 3.27
GSM8K	q4: naive	47.1 ms/tok		eagle	19.1 ms/tok		τ 3.68
HumanEval	q0: naive	46.7 ms/tok		eagle	16.3 ms/tok		τ 4.29
HumanEval	q1: naive	47.7 ms/tok		eagle	20.4 ms/tok		τ 3.50
HumanEval	q2: naive	47.1 ms/tok		eagle	21.4 ms/tok		τ 3.33
HumanEval	q3: naive	47.0 ms/tok		eagle	17.6 ms/tok		τ 4.02
HumanEval	q4: naive	47.5 ms/tok		eagle	17.9 ms/tok		τ 3.98
Alpaca	q0: naive	47.0 ms/tok		eagle	25.6 ms/tok		τ 2.71
Alpaca	q1: naive	46.9 ms/tok		eagle	22.1 ms/tok		τ 3.11
Alpaca	q2: naive	47.0 ms/tok		eagle	19.0 ms/tok		τ 3.64
Alpaca	q3: naive	47.0 ms/tok		eagle	18.8 ms/tok		τ 3.71
Alpaca	q4: naive	47.0 ms/tok		eagle	25.4 ms/tok		τ 2.77
Russian	q0: naive	47.2 ms/tok		eagle	61.8 ms/tok		τ 1.12
Russian	q1: naive	46.8 ms/tok		eagle	40.3 ms/tok		τ 1.74
Russian	q2: naive	46.2 ms/tok		eagle	59.4 ms/tok		τ 1.17
Russian	q3: naive	48.8 ms/tok		eagle	42.0 ms/tok		τ 1.64
Russian	q4: naive	47.0 ms/tok		eagle	55.4 ms/tok		τ 1.25

Aggregating by set. The per-question spread goes right next to the mean: without it there is no telling whether two sets differ or it is just noise.

	tau	tau_min	tau_max	speedup	sp_min	sp_max	speedup_hf
bench							
MT-Bench	2.910000	2.540000	3.780000	1.970000	1.710000	2.590000	1.740000
GSM8K	3.680000	3.250000	4.430000	2.470000	2.180000	2.970000	2.230000
HumanEval	3.820000	3.330000	4.290000	2.550000	2.200000	2.870000	2.270000
Alpaca	3.190000	2.710000	3.710000	2.150000	1.830000	2.490000	1.930000
Russian	1.380000	1.120000	1.740000	0.940000	0.760000	1.160000	0.840000

All sets combined: τ = 3.00, speedup 2.02x vs naivenerate, 1.80x vs stock generate
in-domain sets: τ = 3.40, speedup 2.28x
Russian set: τ = 1.38, speedup 0.94x



Takeaway. Domain decides everything. On the English sets acceptance length holds near 3.4 and the method better than doubles the speed; on the Russian prompts it drops to about 1.4 and the same machinery starts losing to plain generation. The spread across questions inside a set is visibly smaller than the gap between sets — so it is the domain, not a few unlucky prompts. The lower panel shows the mechanics: speedup is almost linear in acceptance length, and below the break-even point marked on the plot the line dips under 1.

What this means in practice. On the English sets the method delivers a steady speedup; on the Russian ones it *slows generation down*. That set is out of the head's training distribution and in another language at once, and this run cannot separate the two: the head was community-trained on English dialogue, so training data is the likely cause, but confirming it needs an out-of-domain English set or a Russian-trained head, and neither is here. Whatever the reason, off that domain it rarely guesses right, τ sinks toward 1, and the build-tree $\rightarrow$ verify $\rightarrow$ discard cycle degenerates into pure overhead on

top of an ordinary step.

Only it is measured at the *starting* tree, and that tree is deep. At the shape §7 goes on to pick there is no slowdown — §11 runs a subset of the same Russian prompts and comes out slightly ahead, at practically the same acceptance length. So the number above is not a property of the domain alone: it is measured at a deep tree, and at a shallow one the same set behaves differently. Domain sets how much the method can win; depth sets how much it can lose. That is not a guess: §7 runs the same depth sweep on the Russian set, and the curve crosses 1.0 while acceptance length barely moves.

Speculative decoding guarantees the output, not the gain: measure acceptance length on your own prompt distribution instead of quoting the paper's.

6. The draft tree: anatomy of a single step

Time to look at one tree whole.

Tree shape is set by three parameters:

- `top_k` sets the width: how many continuations open at each node;
- `depth` sets how many expansion steps follow the first draft pass: a cycle costs `depth` + 1 draft launches, and the tree carries `depth` + 1 levels;
- `total_token` sets the budget: how many of all opened nodes stay in the tree.

Construction goes like this. The draft model runs once, then takes `depth` more steps; at each step every one of the top-k current nodes opens its own top-k continuations, forming a candidate pool. Each candidate carries a cumulative log-probability — the sum along its whole path from the root. The `total_token` best by that sum are selected from the pool, and the tree mask and position indices are built over them. The dynamic selection is EAGLE-2's contribution: in EAGLE-1 the tree shape was fixed in advance, blind to context.

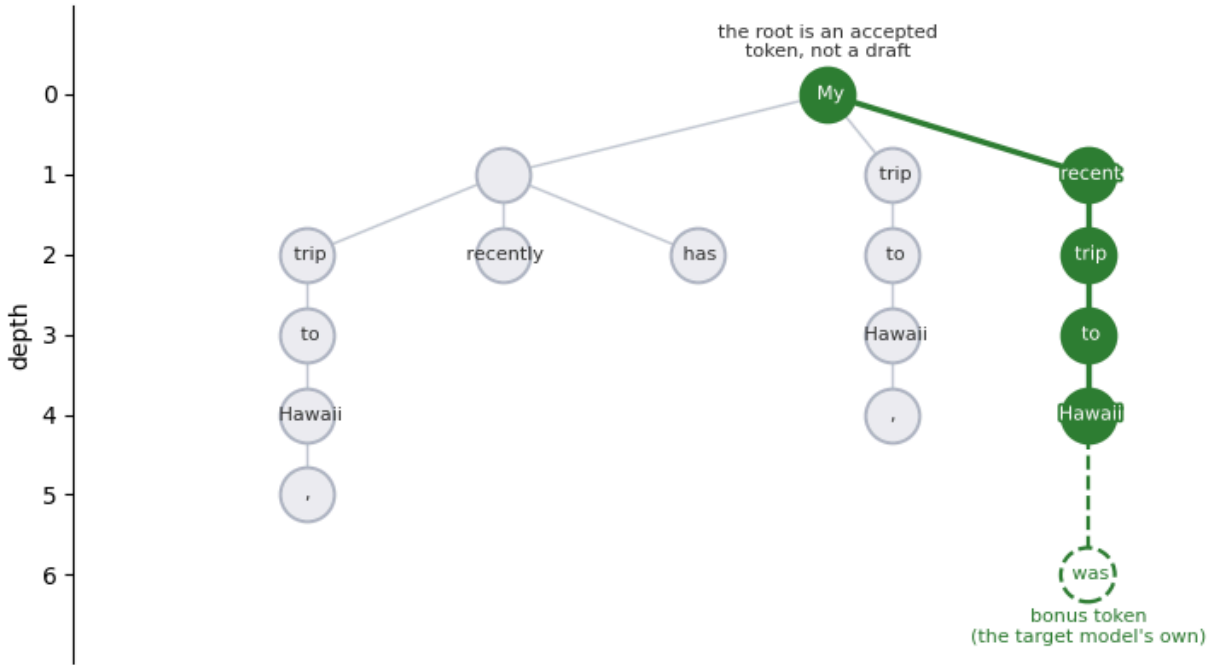
The target model verifies the whole tree in one pass with the tree mask from §2. All root-to-leaf paths are read out of its logits; under greedy decoding a path is accepted exactly up to the position where a draft token stops matching the target's `argmax`. A **bonus token** is appended to the accepted path — the one the target itself predicted at the last accepted position. So even at zero acceptance the cycle emits one token, like plain generation: in target-model passes, the method cannot do worse.

Below is an instrumented copy of the loop: same logic, plus a record of the tree and the verification result at every step.

43 steps, 98 tokens emitted, $\tau = 2.28$

accepted per cycle: [2, 1, 3, 3, 2, 2, 1, 3, 1, 4, 2, 1, 3, 2, 1, 2, 1, 3, 1, 2, 2, 4, 3, 2, 2, 2, 5, 2, 2, 1, 3, 1, 5, 2, 1, 1, 3, 3, 5, 1, 4, 2, 2]

A 16-node tree: 4 drafts + 1 bonus = 5 tokens accepted in a single target-model pass



Paths of this tree (first 6):

[rejected] My $\rightarrow$ $\rightarrow$ recently

[rejected] My $\rightarrow$ $\rightarrow$ has

[ACCEPTED] My -> recent -> trip -> to -> Hawaii

```
[rejected] My -> trip -> to -> Hawaii -> ,
```

[rejected] My -> -> trip -> to -> Hawaii -> ,

Takeaway. Here is what separates a tree from a chain: the draft proposed several mutually exclusive continuations, and the target verified them all in one pass. The green path matched its own choice and was accepted whole; the other branches were dropped. The last token of the path comes from the target itself, which is why even a complete miss leaves the cycle no worse than an ordinary step.

What branching itself buys

Two things, looked at separately.

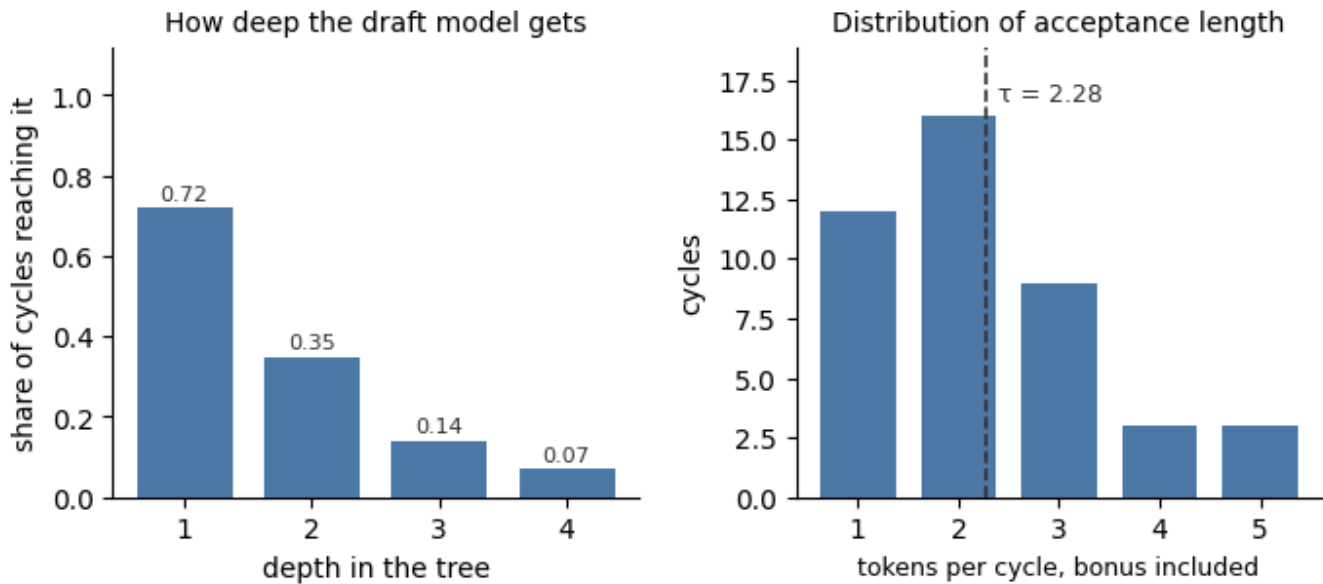
First, acceptance by depth: the share of cycles that survived to depth 1, 2, 3 and so on. It shows exactly where the draft model runs out of steam.

Second, how much the branching adds. The degenerate tree at `top_k = 1` is classic chain speculative decoding: the draft proposes one sequence and the target checks it. Chain and tree are compared at the same depth, on the same prompt, with the same number of target-model passes.

tree τ (16/5/3): 2.28 tokens per cycle

chain τ (6/5/1): 1.64 – same draft model, same depth, no branching

=> branching adds 39% to acceptance length at the same number of target-model passes



Takeaway. Acceptance falls off fast: only 72% of cycles get even one draft token accepted, 35% reach two, and 7% reach four — the deepest any cycle got here. The next section confirms this with a proper sweep, but the conclusion is visible already: growing the tree beyond a few levels is pointless, because depth is paid for every time and pays back rarely.

7. Tree shape: two one-knob sweeps

§1 predicts an asymmetry. **Width** means extra nodes inside the same verification pass, which stays flat to about 192 tokens — so width is almost free. **Depth** means additional sequential launches of the draft model, each with a fixed price nothing can hide.

To test this, one knob moves at a time: first `total_token` is fixed and depth sweeps, then depth is fixed and `total_token` sweeps. Every point is measured three times, and the plot shows mean and spread — without that, picking a configuration on a couple-percent difference is meaningless, since run-to-run GPU noise is the same order.

The cell below runs both sweeps, printing mean, spread and acceptance length for every point.

plain generation: 46.85 ± 0.45 ms/token (spread 1.0%)

A. depth sweep at fixed total_token = 64, top_k = 10

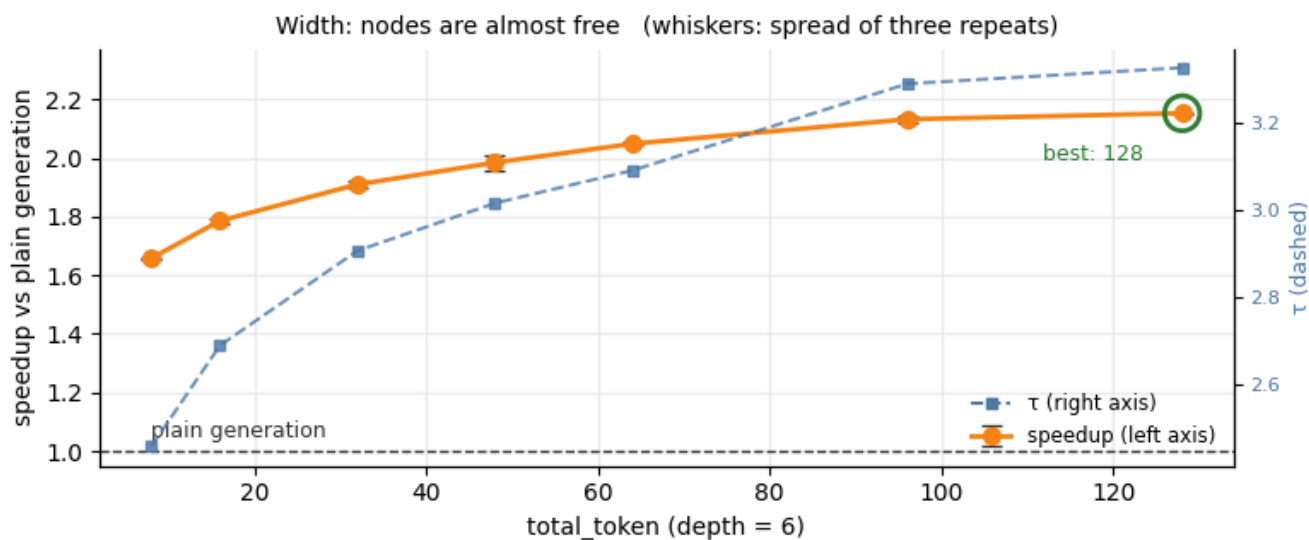
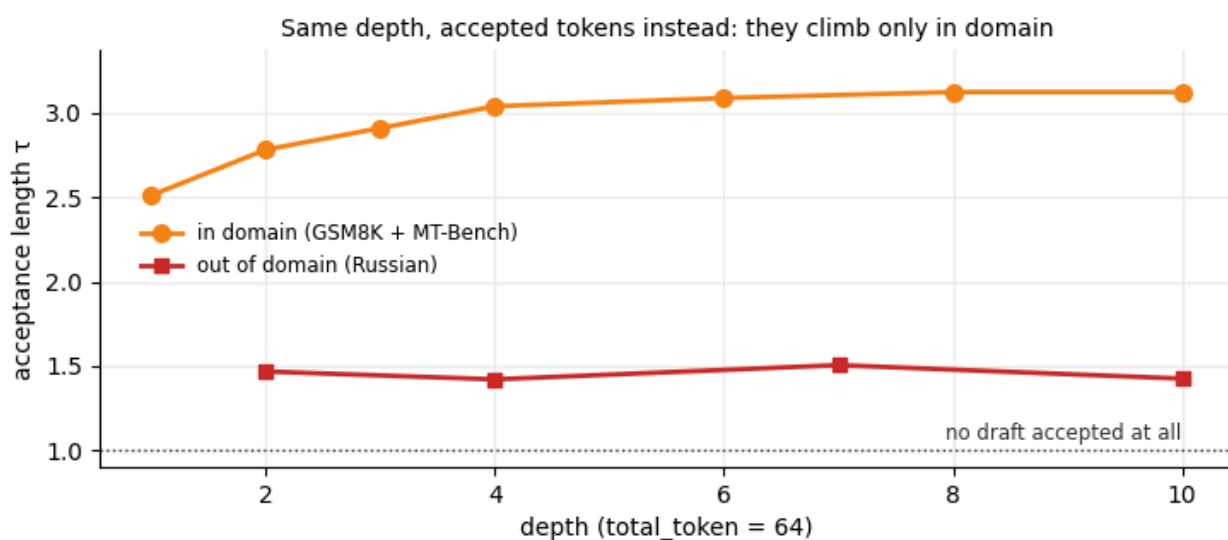
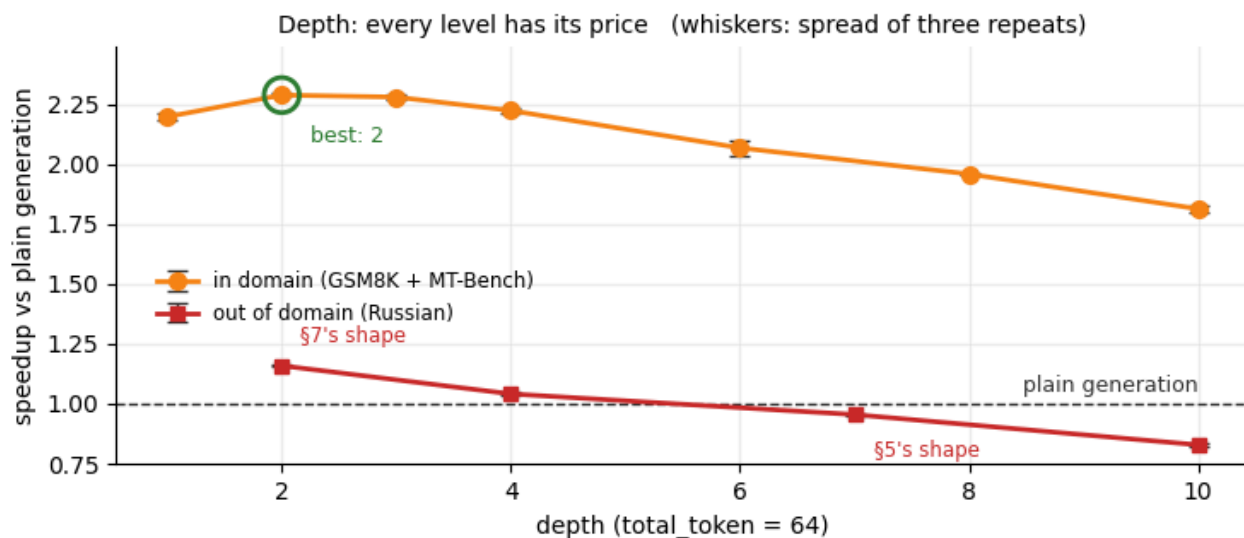
depth= 1:	21.32 ± 0.12 ms/tok	τ 2.51	speedup 2.20x
depth= 2:	20.48 ± 0.07 ms/tok	τ 2.78	speedup 2.29x
depth= 3:	20.55 ± 0.08 ms/tok	τ 2.91	speedup 2.28x
depth= 4:	21.07 ± 0.13 ms/tok	τ 3.04	speedup 2.22x
depth= 6:	22.66 ± 0.35 ms/tok	τ 3.09	speedup 2.07x
depth= 8:	23.92 ± 0.06 ms/tok	τ 3.12	speedup 1.96x
depth=10:	25.84 ± 0.25 ms/tok	τ 3.12	speedup 1.81x

C. depth on the out-of-domain set (plain generation 46.77 ms/tok)

depth= 2:	40.36 ± 0.19 ms/tok	τ 1.47	speedup 1.16x
depth= 4:	44.92 ± 0.29 ms/tok	τ 1.42	speedup 1.04x
depth= 7:	48.97 ± 0.18 ms/tok	τ 1.50	speedup 0.96x
depth=10:	56.44 ± 0.59 ms/tok	τ 1.42	speedup 0.83x

B. tree-size sweep at fixed depth = 6, top_k = 10

total_token= 8:	28.26 ± 0.05 ms/tok	τ 2.46	speedup 1.66x
total_token= 16:	26.23 ± 0.13 ms/tok	τ 2.69	speedup 1.79x
total_token= 32:	24.53 ± 0.12 ms/tok	τ 2.91	speedup 1.91x
total_token= 48:	23.60 ± 0.30 ms/tok	τ 3.01	speedup 1.98x
total_token= 64:	22.86 ± 0.08 ms/tok	τ 3.09	speedup 2.05x
total_token= 96:	21.97 ± 0.13 ms/tok	τ 3.29	speedup 2.13x
total_token=128:	21.76 ± 0.05 ms/tok	τ 3.32	speedup 2.15x



best depth (at total_token = 64): 2
best size (at depth = 6): 128
 τ as the tree grows: 2.46 $\rightarrow$ 3.32, as depth grows: 2.51 $\rightarrow$ 3.12

combined point 128/2/10: 20.79 $\pm$ 0.23 ms/tok | τ 2.84 | speedup 2.25x
it is not better than the best single-sweep points – the knobs interact; taking the best verified configuration

Takeaway. The two sweeps behave differently, and exactly as the arithmetic of §1 predicts. Depth has a clear optimum past which it only hurts: acceptance length plateaus while time keeps growing with every extra draft-model launch. Width is nearly free while the pass stays flat, so its curve climbs to a ceiling instead of collapsing.

The red line on the top panel is the same depth sweep on the out-of-domain set, and it explains §5's slowdown. The middle panel shows why: acceptance length there stays flat, near one and a half at every depth, while in domain it climbs with every level. Off its own domain the draft guesses just as rarely at two levels as at ten. So depth buys nothing there but price, and on a deep tree the price wins: the top curve crosses under 1.0. So §5, which measures at the deep starting shape, finds a slowdown, while at the shallow shape picked here the same prompts come out slightly ahead. Domain decides how much the method can win; depth decides how much it can lose, sign included.

The shallow depths are separated by a couple of percent, which is about the drift between whole sessions — and across runs of this notebook the winner has moved between depth 2 and depth 3 while the shape of both curves stayed put. So read the sweep for its shape, not for whichever point came out best: what it establishes is what the deep default costs: about 10%. Which of the shallow depths wins it does not settle.

The library ships a mode that picks the tree size on its own. Comparing its choice with the sweeps above.

the authors' auto-pick would choose total_token = 60
our width sweep peaks near 128, but it runs at depth 6 – and at the depth this section actually picks (2), width saturates far earlier: the combined point 128/2/10 is measured against the single-sweep winners below, and the knobs turn out to interact.

The gap that matters is the knob the heuristic never turns. It searches total_token in 40..60 and leaves depth fixed, while on this card depth is what costs: every level adds a sequential draft launch, and acceptance plateaus by level four.

Proceeding with the tree {'total_token': 64, 'depth': 2, 'top_k': 10}

Takeaway. The authors' auto-pick tunes the one knob that does not decide anything here. It searches node count in the range 40–60 and never touches depth — and depth is what costs on this card: the shape this section settles on carries 64 nodes, a mere four above the heuristic's ceiling, while dropping depth from the loaded default is worth about 10%. The width sweep's own optimum sits far higher than either, but it is measured at depth 6 and does not survive the move to a shallow tree — the combined

point printed above lost to the winner of the depth sweep. The heuristic was tuned on other models and other hardware — exactly the case where a borrowed setting should be re-measured, not inherited.

8. Where the cycle spends its time

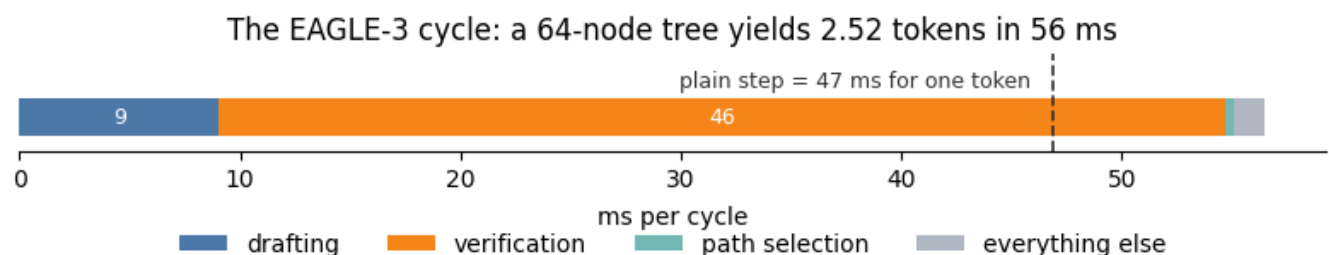
A cycle has four phases: drafting (sequential head passes), verification (one target pass over the whole tree), path selection, and everything else — cache copying, building the next tree, host-side bookkeeping.

Synchronizing wrappers cost time themselves, so "everything else" is not computed by subtraction alone: the same run is timed again without instrumentation, and the difference between the two runs is the price of measuring — shown, not hidden.

Wrapping the three phases in timers and running twice: instrumented and clean.

```
cycle (instrumented): 56.4 ms, yields  $\tau = 2.52$ 
cycle (clean):       56.3 ms <- measurement overhead 0.1 ms
  drafting           9.0 ms (16.0%)
  verification       45.6 ms (80.9%)
  path selection      0.4 ms ( 0.7%)
  everything else     1.4 ms ( 2.5%)
```

```
plain step: 46.8 ms per token
verifying a 64-node tree: 45.6 ms = 0.97 of a plain step
and the §1 target pass at 64 tokens: 47.7 ms — they agree
```



Takeaway. Verification takes about 80% of the cycle and costs almost exactly what a plain step costs for a single token — except it brings back two and a half, the tree now in the shape §7 selected — the figure title above names it. The draft model takes about 15%, and path selection together with cache copying comes to a couple of percent. Same price, more product.

9. Temperature: where the method's main guarantee is tested

The other half of the promise is that the method spoils nothing.

Under greedy decoding "no quality loss" is nearly a tautology: only a draft matching the target's `argmax` gets accepted. The substantive guarantee is stated for sampling: speculative decoding must produce exactly the same distribution as ordinary sampling from the target model.

Precision matters here, because the repository's implementation differs from the classic papers. In [Leviathan](#) and [Chen](#), a draft is accepted with probability $\min(1, p/q)$, where p is the target's probability and q the draft's, and on rejection one samples from the normalized difference. The EAGLE code (`evaluate_posterior`, the `logits_processor` branch) sets `qx = 1.0`: a candidate is accepted with probability $p(x)$; on rejection $p(x)$ is zeroed out, the distribution renormalized, and the next unique candidate from the tree is tried.

This scheme is unbiased too, and one line shows it. The probability that candidate a is rejected and token b lands in its place is

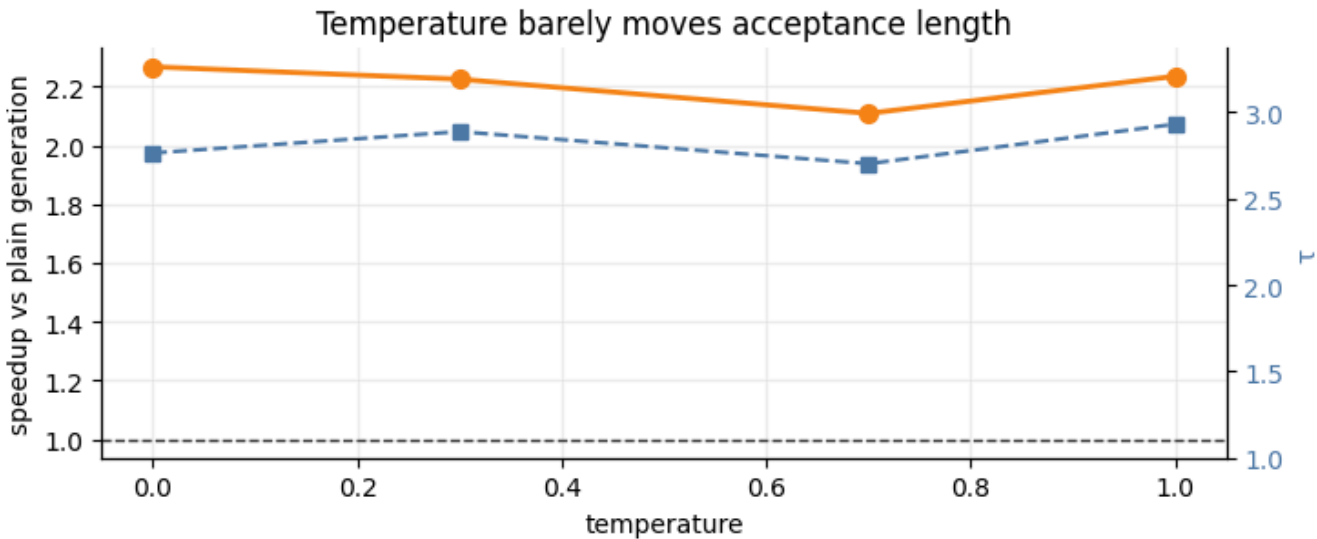
$$(1 - p(a)) \cdot \frac{p(b)}{1 - p(a)} = p(b)$$

The first factor is the probability of rejecting the candidate, the second of picking b from the renormalized distribution; the rejection factor cancels against the normalization. Induction over candidates extends this to any number of them, so the final distribution equals p under any proposal strategy: the draft model affects speed, never the outcome.

The proof lives in exact arithmetic. In fp16, tree logits and single-token logits are computed in different operation orders and need not agree bit for bit. So both things get measured: whether the distributions match, and by how much the logits actually drift.

Running the same prompts at four temperatures, watching acceptance length and speed.

T=0.0: τ 2.76 | 20.66 ms/tok | speedup 2.27x
T=0.3: τ 2.89 | 21.05 ms/tok | speedup 2.23x
T=0.7: τ 2.70 | 22.21 ms/tok | speedup 2.11x
T=1.0: τ 2.93 | 20.96 ms/tok | speedup 2.24x



Takeaway. The expectation was: hotter sampling, rarer draft hits, smaller gain. The authors confirm it — for LLaMA-3.1-8B they report 4.44× at T = 0 against 3.45× at T = 1, roughly a fifth lost. My measurement shows no such drop: acceptance length and speedup wander within a few percent with no clear trend. I would not call that a refutation — this is two prompts without repeats, a target almost five times smaller, and shorter generations, so the sensitivity may simply not be there. What can be claimed: full sampling causes no collapse, and the method stays roughly twice as fast as plain generation.

Collecting 500 independent continuations per loop. Their random-seed ranges differ on purpose: the samples must be independent, or the comparison becomes paired and shows agreement where there is none.

collected in 500 s: EAGLE 500, plain 500, control 500

Statistics are easy to ruin here, so a word about the test.

Comparing empirical distributions by total variation distance (TV) is not enough on its own: two finite samples from the very same distribution still have TV above zero, and the wider the support, the larger it gets. What is needed is a null band — how much TV accrues merely because samples are finite.

Building that band by bootstrapping the observed sample is a trap: resampling from a pool produces more coincidences than two genuinely independent draws, the band comes out too narrow, and the test declares any difference significant. The right tool is a **permutation test**: if the distributions match, the sample labels are exchangeable, so shuffling labels across the pooled samples yields an exact null with no assumptions about the support.

And to avoid judging the test's health by the very data it checks, a control runs alongside: two independent samples of plain generation. The test must find no difference there.

CONTROL — plain vs plain (the test must find NO difference):

joint distribution of 5 tokens: TV=0.1440, threshold=0.1940, p=0.865 test is healthy

TEST — EAGLE vs plain:

joint distribution of 5 tokens: TV=0.2060, threshold=0.2060, p=0.053

distinct sequences: plain 65, EAGLE 65

VERDICT: distributions indistinguishable

per position (position 0 is drawn straight from the target's logits, positions 1 and later pass through speculative accept/reject):

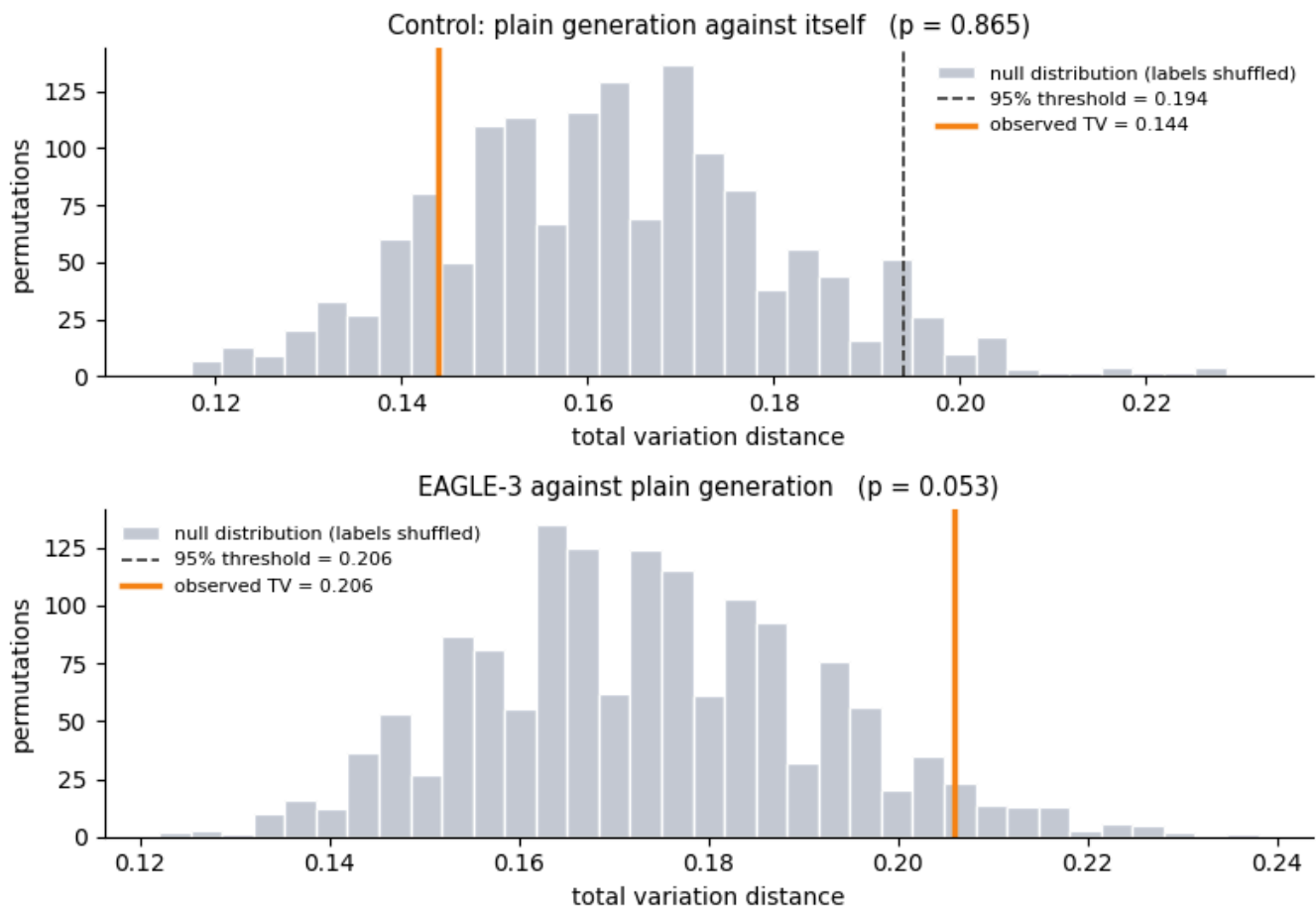
pos 0: EAGLE p=0.033 !! | control p=0.827 | distinct tokens 10

pos 1: EAGLE p=0.528 ok | control p=0.410 | distinct tokens 3

pos 2: EAGLE p=0.003 !! | control p=0.192 | distinct tokens 12

pos 3: EAGLE p=0.498 ok | control p=0.482 | distinct tokens 11

pos 4: EAGLE p=0.118 ok | control p=0.507 | distinct tokens 16



first-token frequencies (plain | control | EAGLE):

'book':	174	174	160
'apple':	137	141	180
'dog':	72	83	65
'cat':	68	60	51
'car':	35	27	29
'bird':	6	4	6
'tree':	3	6	4
'door':	3	2	1

Takeaway. Top panel, the control: two independent samples of plain generation, and the test finds no difference — so it is not inclined to see gaps where there are none. Bottom panel, the EAGLE comparison: the observed distance sits inside the same null distribution. It sits tight, though — p comes out near the boundary, and this test leaves no margin. The question is settled not by it but by the direct distribution comparison below.

Where the residual difference comes from

The joint test finds no difference, yet two positions out of five come back with low p. Before declaring the guarantee broken, consider position 0: the first new token is drawn straight from the target's logits in both loops — the speculative accept plays no part in it. If a difference exists already there, its source is arithmetic, not the acceptance rule.

This can be checked exactly, no statistics: intercept the probability tensor each loop draws its first token from and compare the two distributions head-on.

The cell below hooks the token draw and prints both probability tables side by side.

```
vocabulary size: 151936
TV between the first-token distributions: 0.00000
largest single-token probability difference: 0.00000
bit-for-bit identical: True
```

token	plain	EAGLE	delta
'apple'	0.33350	0.33350	+0.00000
'book'	0.31812	0.31812	+0.00000
'dog'	0.15259	0.15259	+0.00000
'cat'	0.11169	0.11169	+0.00000
'car'	0.05792	0.05792	+0.00000
'tree'	0.01072	0.01072	+0.00000
'bird'	0.00714	0.00714	+0.00000
'door'	0.00302	0.00302	+0.00000

The distributions match bit for bit – the residual difference in the test above is explained by finite samples and multiple comparisons.

10. Matching plain generation at $T = 0$

Back to greedy mode, where the requirement is stricter: not the distribution but the sequence itself must match. EAGLE must produce the exact same token chain as the plain loop. The outputs are compared token by token — and if divergences turn up, I measure how large they are and where they arise.

The hypothesis is simple. A plain step processes one token; a speculative step, a tree of dozens — different tensor shapes, different CUDA kernels, different fp16 summation order. If at some position the two best candidates are nearly tied in logit, summation order decides the outcome. So at divergence points the top-1/top-2 gap should be anomalously small compared to ordinary positions. That is a testable prediction, and easy to refute: if the gaps at divergence points look ordinary, the cause is the implementation, not numerics.

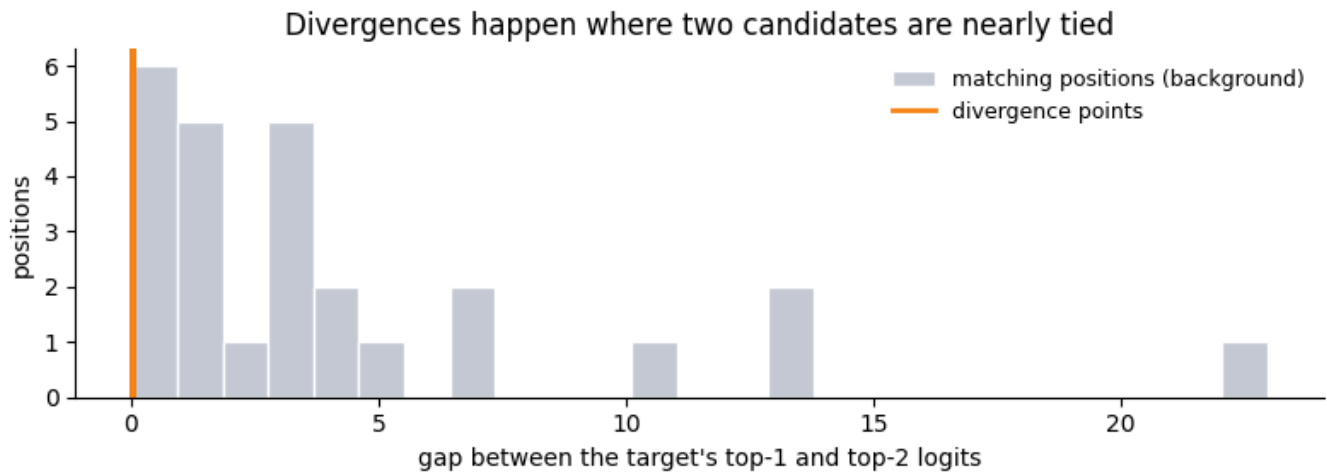
Comparing the two outputs token by token across ten prompts.

```
checked 10 prompts, 2219 tokens
perfect match: 6 of 10 prompts
divergences: 4 = 1.80 per 1000 tokens
```

Now the hypothesis itself: take the positions where outputs diverged and look at the gap between the target's two best candidates, with the same gap on matching positions as the background.

prompt: MT-Bench – 'Compose an engaging travel blog post about a recent trip to '
178 tokens matched, then diverged
plain: ' left me in tears of appreciation.\n\nI'
EAGLE: ' truly brought me into the heart of Hawaiian'

top1-top2 gap at the divergence point: 0.0000
median gap at matching positions: 2.8359
divergence point's percentile in the background: 0%



Conclusion: the gaps at divergence points hug zero while an ordinary position is confidently separated. This is exactly the picture the fp16 summation-order hypothesis predicts, not a flaw in the acceptance rule: the losslessness guarantee is stated for exact arithmetic.

Takeaway. The hypothesis was falsifiable: if divergences come from numerical noise, they have to fall on positions where two candidates are nearly indistinguishable. So they did — the divergence points hug zero while an ordinary position keeps a confident margin. A consequence of fp16 summation order, not a defect of the acceptance rule.

11. The same method, a bigger target

The formula suggests a prediction this notebook has not tested yet. The usual reasoning goes: a draft head is small and roughly fixed in cost, while the target's step grows with model size, so the same method should pay better on a bigger target. That reasoning is checkable, and the check is worth running before repeating it. Qwen3-4B is the largest pair that fits a free T4 in fp16 (8B weighs 16.4 GB, more than the card holds). In the other direction there is simply nothing to compare against: AngelSlim publishes no head for Qwen3-0.6B, so 1.7B is the small end of what can be measured here rather than a floor I picked.

So: the 1.7B pair is re-measured on a small subset at the tree shape §7 picked, then leaves the GPU entirely; the 4B pair loads from its own attached models and runs the same subset, same tree, same equal-work rules.

Before the numbers, one thing about the run itself. The official repository cannot load this head as published: Qwen3-4B's attention is wider than its hidden size ($32 \text{ heads} \times 128 = 4096$ against 2560), while the official draft code derives head width from the hidden size alone. The environment cell patches three lines of `cnets.py` to honor the config's `head_dim` — a no-op for the 1.7B pair and the only modification to the official code in this notebook.

1.7B pair at `{'total_token': 64, 'depth': 2, 'top_k': 10}`:

in : naive 46.5 ms/tok | eagle 19.2 | τ 2.96 ± 0.00 | speedup 2.42x ± 0.03 over 3 repeats

out: naive 46.2 ms/tok | eagle 42.3 | τ 1.36 ± 0.00 | speedup 1.09x ± 0.01 over 3 repeats

GPU after unload: 0.01 GB allocated

/kaggle/input/models/qwen-lm/qwen-3/transformers/4b/1

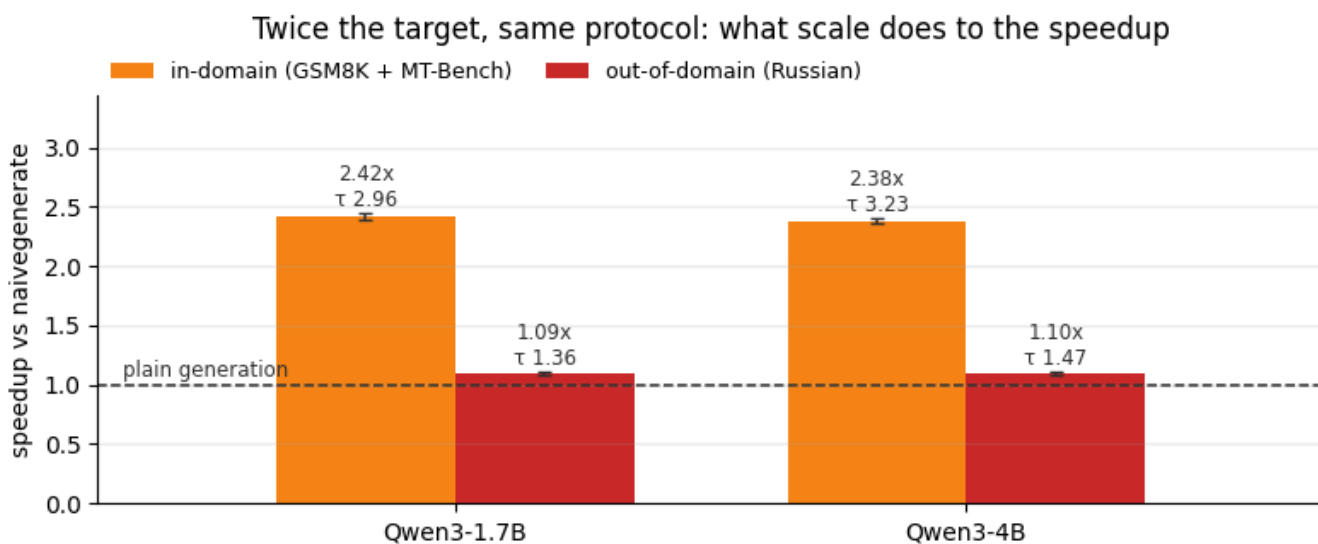
/kaggle/input/models/georgymamarin/qwen3-4b-eagle3-draft-head/pytorch/bf16/1

4B pair loaded in 68 s | target 4.02B params | GPU 8.65 GB

4B pair at `{'total_token': 64, 'depth': 2, 'top_k': 10}`:

in : naive 60.2 ms/tok | eagle 25.3 | τ 3.23 ± 0.00 | speedup 2.38x ± 0.02 over 3 repeats

out: naive 60.9 ms/tok | eagle 55.4 | τ 1.47 ± 0.00 | speedup 1.10x ± 0.01 over 3 repeats



Takeaway. Two targets, one protocol, and the prediction only half survives.

Acceptance length does rise with target size, and by a remarkably stable amount: about 9% higher on the 4B pair, in domain and out, on every full run of this notebook — including runs where §7 settled on a different tree and every absolute number shifted with it. Greedy decoding fixes the token sequence, so within a run the three repeats return the same acceptance length to three decimals; the whiskers in the figure above are pure timing jitter. The speedup does not follow, and *how* it fails to follow is the interesting part: across those runs the 4B pair landed anywhere within about 3% either way, the sign changing from run to run. The quantity that reproduces says the draft head got better; the quantity that drifts cannot tell whether anything

changed at all. Doubling the target bought acceptance length and, as far as this card can resolve, no speed.

The mechanism is in the raw timings rather than the ratios, and it needs the cycle and the token kept apart. An ordinary step gets dearer on the bigger target, and a full speculative cycle — one round of drafting plus one tree verification — gets dearer faster still, on every run: verification carries the same widened weights the step does, while drafting is no longer free. The 9% extra tokens each cycle returns then buys back almost exactly that difference. What survives the cancellation is the couple of percent above, which is the same size as the drift between runs — hence the moving sign. The cell's output prints the run's own figures; the relation between them is what holds.

So it is the premise that fails: a draft head is not fixed in cost. This one grew with its target, from 137M parameters to 218M, because its hidden size and attention widen alongside the model. It does shrink as a share of the target, from 8.0% to 5.4%, but share is not what the formula's denominator charges for — absolute draft time is.

The out-of-domain bars point at the other open question without closing it. Neither pair falls below 1.0 here, on any run — so the win itself is not in doubt. What is in doubt is the cause. This subset differs from §5's in more than the tree shape — fewer prompts, a shorter budget — so on its own it cannot say which of those changes lifted the Russian set from 0.95x to above 1.0. One thing it can say: acceptance length barely moved between the two, so the shorter subset is not an easier one, and what changed is the price of the cycle; how often the draft is right stayed put. That points at tree depth, and §7 tests the guess directly: it sweeps depth on the Russian set too, and the curve crosses 1.0 while acceptance length holds near one and a half throughout. What the run does support is the ordering: domain sets how much speculation can win, tree shape sets how much it can lose.

key_numbers.json written (15 groups, scaling: yes)

12. What didn't work

Dead ends and traps that cost time, listed so they are not repeated.

P100 instead of T4. Kaggle's default under `enable_gpu` is a P100, and the preinstalled `torch 2.10+cu128` does not support `sm_60`: it dies with `CUDA error: no kernel image is available`. The fix is picking a T4 (via the API, `machine_shape: NvidiaTeslaT4`).

Installing the repo through pip. EAGLE's `setup.py` pulls `torch==2.0.1` and `transformers==4.46.2`, which wrecks a fresh environment. The right path: clone, add to `sys.path`, and pin `transformers` separately (`4.53.1`; the preinstalled 5.x is incompatible with the repo's modified modeling files).

The deep default tree. The starting shape `60/7/10` and the authors' `32/8/4` share one trait: depth 7 and 8. The §7 sweep shows depth does not pay on this card — acceptance plateaus by level

four while every level bills. I did not measure `32/8/4` itself: the §7 depth sweep moves depth at fixed `total_token` and `top_k`, touching one knob of three, so I quote no numbers for that exact shape — the conclusion is about depth, not one configuration.

Chasing percent-level differences. An early version picked its configuration on 2.42x vs 2.43x from a single run per point. Repeats showed about a percent of run-to-run spread: the difference was noise. Hence the repeats (three per point) and whiskers on every plot.

Growing the tree forever. τ keeps rising past 96 nodes; speed does not — the gain is eaten by the cost of building the tree itself. The bottleneck migrates from verification to the draft model.

A bootstrap null band. The first $T = 1$ test built its null band by resampling the observed sample. On a sparse support this systematically understates the band: a bootstrap draws from a pool of a few hundred values and produces more coincidences than two truly independent draws. The test confidently declared a difference where none exists. A permutation test does not have this problem, and the plain-vs-plain control catches the failure instantly: on the broken test the control showed a difference too.

Chasing low per-position p-values. Even with a sound test, two positions of five came back with $p < 0.05$. Tempting to declare the guarantee broken, but with ten comparisons (five positions, two tests each) at $\alpha = 0.05$ the expectation is half a false positive on average, so p-values alone do not settle it. For position 0 a direct measurement did: the distributions match bit for bit, leaving the difference nowhere to come from. For later positions a direct comparison is impossible in principle, since the distribution there depends on the already-sampled prefix, which each run grows on its own. So those p-values go down to multiple comparisons: nothing further is provable here.

The official loader rejects the 4B head. `AngelSlim/Qwen3-4B_eagle3` is listed in the official weights table, yet `EaModel.from_pretrained` dies on it with state-dict size mismatches: the head was trained with `head_dim = 128` at 32 heads (attention width 4096, hidden 2560 — the modern Qwen3 layout), and the official `cnets.py` hardcodes `head_dim = hidden // heads`. True on the pinned commit and on current `main`. The three-line fix lives in the environment cell; without it, only AngelSlim's own framework or vLLM can run this head.

Testing the distribution on a "convenient" prompt. The first attempt to check distribution invariance at $T = 1$ used a prompt whose continuation is nearly deterministic: TV came out exactly 0 against a null band of $[0, 0]$. Formally a match; substantively, the test checked nothing. The prompt needs real entropy, or emptiness is being verified.

13. Takeaways

1. **Speculative decoding works because decoding is memory-bound.** On a T4 the target pass barely gets dearer up to about 192 tokens, so verifying a whole tree costs about what generating one token costs. Everything else in the method is an attempt to pack more accepted tokens into that single pass.

2. **Speedup is set by acceptance length, and acceptance length by domain and tree shape together.** The same draft head spans two-and-a-half-fold gains on math and code down to an outright slowdown on Russian prompts — but that slowdown is measured at the deep starting tree. At the shape §7 picks, the same prompts land above 1.0 at practically the same acceptance length, and §11 sees the same on a second pair of models. Domain sets how much the method can win; tree depth sets how much it can lose. Measure acceptance length on your own prompt distribution; other people's numbers do not transfer.
3. **Tree width is cheaper than depth.** Width adds nodes to the same verification pass; depth adds new sequential draft-model launches. Meanwhile the authors' auto-pick heuristic tunes node count and never turns the depth knob at all.
4. **The method does not spoil the output, verifiably in three ways.** At $T = 0$ the output matches token for token, and the rare divergences land strictly on positions where two candidates are nearly tied in logit: the gap there is indistinguishable from zero against a typical 2.8 logits. At $T = 1$ a permutation test does not reject equality of distributions, and a plain-vs-plain control shows the test finds no gap where none can exist — on the earlier, broken test that same control showed one. The third way is direct comparison: the distribution both loops draw their first token from matches bit for bit. Later positions cannot be compared this way — their distribution depends on a prefix each run grows on its own.
5. **Claimed speedups must always be read together with their baseline.** The plain step here runs at a quarter of the memory bandwidth; part of the measured gain amortizes that overhead rather than beating the memory wall. On a tuned stack the same method yields less — exactly why independent vLLM measurements are more modest than the authors'.
6. **Scale buys acceptance length, not speed — at least across this step.** The intuition that a bigger target amortizes the draft head better is checkable, and §11 checks it on Qwen3-4B: τ rises about 9% and the speedup does not move outside the noise, because this head family grows with its target instead of staying fixed. Whether the trend resumes at 8B and beyond is open — that pair does not fit a free T4 in fp16.

What is not here. Batches above one (the repository implementation only does batch = 1, and batching is what erodes speculation most), long contexts, and training your own head for your domain. The last is the most interesting follow-up, given point 2.

14. Run it yourself, further reading, licenses

Fork the notebook and change two of the four lines described in §0:

```
BASE_MODEL = "Qwen/Qwen3-4B"           # any target model that has a
head
EA_MODEL    = "AngelSlim/Qwen3-4B_eagle3" # heads are listed in the EAGLE
README
```

On size, since the example above is the case that hits it. §5 loads a second, independent copy of the target alongside the EAGLE pair to time the stock `generate`, so the two-line fork is validated up to roughly 2B on a T4. The 4B pair alone takes most of the card (§11 prints the exact figure), and the extra copy does not fit on top: for a target this size, either drop the stock- `generate` baseline in §5 or unload the first pair before loading the second, the way §11 does.

To measure acceptance length on your own prompts (takeaway 2), add your set to the `BENCH` dict in §5 — every table and figure downstream rebuilds around it. Where the break-even sits depends on the tree, so carry over the shape rather than the number: the §5 plot fits it near one and a half tokens per cycle at that section's deep starting tree, while at the shallow tree §7 settles on, §11 measured a small win at acceptance length about 1.5. What travels is the ordering §11 measured: domain sets how much speculation can win, tree shape sets how much it can lose. Whether a loss *requires* both is untested — the one run at the shallow tree also changed the prompt set and the budget.

The cell below prints the current configuration and what is worth changing in it.

```
# The current configuration. Changed in the notebook's first cell.
print(f'BASE_MODEL = "{BASE_MODEL}"')
print(f'EA_MODEL   = "{EA_MODEL}"')
print(f'tree      = {MAIN_TREE}')
print(f'sets      = {list(BENCH)}')
print()
print("To run on your model: change the first two lines and Run All.")
print("To run on your tasks: add your own list to BENCH (§5).")

BASE_MODEL = "/kaggle/input/models/qwen-lm/qwen-3/transformers/1.7b/1"
EA_MODEL   = "/kaggle/input/models/georgymamarin/qwen3-1-7b-eagle3-draft-head/pytorc
h/fp16/1"
tree      = {'total_token': 64, 'depth': 2, 'top_k': 10}
sets      = ['MT-Bench', 'GSM8K', 'HumanEval', 'Alpaca', 'Russian']
```

To run on your model: change the first two lines and Run All.
To run on your tasks: add your own list to BENCH (§5).

Further reading

How the method reached its current form

- [Leviathan et al., 2211.17192](#) — the original idea and the unbiasedness proof via rejection sampling; the proof itself sits in Appendix A.1 if you want to check it.
- [Chen et al., 2302.01318](#) — independent parallel work, worth reading for a second angle on the same proof.
- [EAGLE-1, 2401.15077](#) — why predicting a feature is easier than predicting a token.
- [EAGLE-2, 2406.16858](#) — the dynamic tree; `total_token`, `depth` and `top_k` from §7 originate here.
- [EAGLE-3, 2503.01840](#) — the paper under study: dropping `l_fea`, three-level fusion, training-time test.

- [HASS, 2408.15766](#) — the closest ancestor of unrolling the draft on its own outputs during training.

How it looks in production

- "[Performance or Illusion?](#)", [2601.11580](#) — an independent replication on vLLM. The main point: gains shrink as batch size grows, because batching keeps the compute units busy on its own, leaving nothing for speculation.

Code and weights

- [SafeAILab/EAGLE](#) — the official implementation, commit `cb7e0841`, Apache 2.0. This notebook uses it as published, with the single three-line exception §11 documents; `eagle/model/cnets.py` (tree construction) and `eagle/model/utils.py` (verification and KV handling) are the files worth reading.
- [AngelSlim/Qwen3-1.7B_eagle3](#) — the draft head, trained by the [AngelSlim](#) team (Tencent), attached here as a [Kaggle model](#). Not the authors' own: the team's measurements report acceptance length 1.8–3.5, which matches mine; their 1.4–1.9× speedup is quoted against their own baseline.
- [Qwen/Qwen3-1.7B](#) — the target model, Apache 2.0, attached from the official [Qwen 3 Kaggle page](#).
- The question sets (MT-Bench, GSM8K, HumanEval, Alpaca) come from the same repository's `eagle/data` — the very sets the paper reports on.

Прочсть этот разбор по-русски: [русская версия](#) — исходный ноутбук, сделанный для отбора Школы анализа данных.

Author and credits

Written by [Georgy Mamarin](#): the questions, the measurements, the notebook code and the text. The method, the implementation, the draft-head weights and the question sets belong to the authors listed above.

Particular thanks to the [SafeAILab](#) team, whose open implementation this walkthrough runs as published, save for the three lines §11 needs to load a head with a modern `head_dim` and documents in place; to the [AngelSlim](#) team at Tencent for the public draft heads for Qwen3-1.7B and Qwen3-4B; and to Kaggle for the free T4 everything here was computed on.

The notebook's code is Apache 2.0; models and weights keep their own licenses.

If you reproduce this on another model or catch a measurement error, say so in the comments — both are equally useful news.